

[41] Filtered-DiskANN; Graph algorithms for approximate nearest neighbor search with filters

저자: S. Gollapudi et al.

학술지: WWW '23, p. 3406–3416

주제: DiskANN 기반의 필터링된 벡터 검색

요약

본 논문은 속성 필터링을 고려한 효율적인 근사 근접 이웃 검색 알고리즘 Filtered-DiskANN 을 제시한다. DiskANN 은 원래 디스크 기반 대규모 벡터 인덱싱을 위한 알고리즘이며, 본 논문에서는 이를 확장하여 속성별 필터링을 통합한다.

Filtered-DiskANN 의 핵심은 필터 조건을 만족하는 벡터들의 부분집합에 대한 근접 이웃을 효율적으로 찾는 것이다. 필터의 선택도(selectivity)에 상관없이 일정 수준의 성능을 유지하면서도 메모리 효율성을 보장한다.

본 논문과의 관계: Exqutor 의 핵심 문제인 "필터 조건과 거리 기반 검색의 효율적 결합"을 직접 다루는 논문이다. Filtered-DiskANN 의 설계 원칙과 최적화 기법은 Exqutor 의 필터링 전략과 매우 유사하다.

상세분석

41.1 DiskANN 의 배경

DiskANN 의 특징:

- SSD 디스크를 주로 사용하는 저비용 대규모 인덱싱
- 메모리에는 최소한의 구조만 유지
- Vamana 그래프 알고리즘 기반

메모리-디스크 계층:

빠르고 작음	느리고 큼
메모리 (GPU/RAM)	← 디스크 (SSD)
↑	↑
핫 데이터	콜드 데이터
(최근 접근)	(대부분)

DiskANN 의 장점:

- 메모리 제약 극복: 수십억 개 벡터 지원
- 비용 효율적: SSD 가격이 RAM 보다 저렴
- 높은 성능: 벡터당 수 밀리초 검색

41.2 필터링의 도전

기본 문제:

필터 조건 P 가 주어졌을 때, 조건을 만족하는 벡터 $V_P \subseteq V$ 중에서 쿼리 q 에 가장 가까운 K 개를 찾기.

문제: FILTERED_KNN(q, K, P)

입력: 쿼리 q , K , 필터 P

출력: $\{v \in V \mid P(v)\}$ 중 q 에 가장 가까운 K 개

도전:

1. 필터 조건을 만족하는 벡터의 개수 미지수
2. 필터를 만족하는 벡터가 매우 드물 수 있음
3. 필터 조건이 벡터 공간의 구조와 무관

필터 선택도의 영향:

선택도 = $|V_P| / |V|$

선택도 99% (거의 모든 벡터 선택):

- 기본 ANN 검색과 거의 동일
- 작은 오버헤드

선택도 1% (대부분 필터링 제외):

- 넓은 검색 범위 필요
- 높은 계산 비용

선택도 0.1% (매우 선택적):

- 극도로 넓은 검색 필요
- 거의 선형 탐색 수준

41.3 Filtered-DiskANN 의 설계 원칙

필터-거리 통합 접근:

1. 필터-우선 전략 (Filter-First)

1. 필터 조건 만족하는 벡터들 식별
2. 해당 벡터들 중 KNN 검색

장점: 필터 조건 만족 보장

단점: 선택도 낮으면 검색 범위 커짐

1. 하이브리드 전략 (Hybrid)

1. 넓은 범위에서 후보 수집
2. 필터 조건 적용하여 정제
3. 부족한 결과는 추가 검색

장점: 모든 선택도에서 균형

단점: 구현 복잡도 높음

41.4 Filtered-DiskANN 알고리즘

인덱스 구조:

Vamana 그래프:

- 각 벡터가 노드
- 각 노드는 M 개의 이웃 연결
- 거리 기반의 엣지 가중치

메타데이터:

- 각 벡터의 속성값 저장
- 속성 인덱스 (속성 값 → 벡터 ID)
- 선택도 통계

검색 프로시저:

```

procedure FILTERED_SEARCH(q, K, filter_predicate,
    num_search_nodes):
    // 1. 진입점 선택 (필터 만족하는 점)
    entry_point = find_entry_point(filter_predicate)

    // 2. 초기 후보 수집
    visited = {}
    candidates = priority_queue() // 거리순 정렬
    result_set = {}

    candidates.push(entry_point)
    visited.add(entry_point)

    // 3. 그리디 검색
    while not candidates.empty() and
        |visited| < num_search_nodes:

        current = candidates.pop()

        // 필터 조건 확인
        if not filter_predicate(current):
            continue // 필터 미충족 스킵

        // 결과 집합 추가
        dist = distance(q, current)
        if |result_set| < K or dist < max_dist(result_set):
            result_set.add((current, dist))
            if |result_set| > K:
                remove_farthest(result_set)

        // 이웃 탐색
        for neighbor in neighbors[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                candidates.push(neighbor)

    // 4. 결과 정렬 및 반환
    return sort_by_distance(result_set)

```

41.5 필터 선택도 적응

동적 검색 범위 조정:

필터 선택도 추정:

$$\text{selectivity} = |\text{결과_필터_통과}| / |\text{방문_노드}|$$

검색 범위 조정:

if selectivity > 50%: // 필터 선택적이지 않음

 num_search_nodes = K * 2

elif selectivity > 10%:

 num_search_nodes = K * 5

elif selectivity > 1%:

 num_search_nodes = K * 20

else:

 num_search_nodes = K * 100 또는 더 큼

41.6 속성 인덱싱 전략

단순 인덱싱:

속성별 비트맵 인덱스:

color="red" → 비트맵 001010...
(해당 벡터는 1, 아니면 0)

장점: 빠른 필터 평가

단점: 메모리 사용량 (속성당 1 비트/벡터)

계층화 인덱싱:

B-트리 기반 속성 인덱스:

범위 필터 (price > 100) 지원
효율적인 범위 쿼리

디스크 I/O 최적화:

자주 함께 필터링되는 속성들을 근처에 배치

41.7 성능 최적화 기법

진입점 최적화:

필터를 만족하는 진입점을 미리 식별:

방법 1: 필터별 진입점 캐싱

`filtered_entry_point[filter_id] = p`

방법 2: 범용 진입점에서 필터 만족점까지 빠른 이동

`entry_point` 에서 시작

→ 이웃 탐색으로 필터 만족점 빠르게 찾기

이웃 구성 최적화:

필터 친화적 이웃 선택:

전통적 Vamana:

각 점 p 의 이웃 = 가장 가까운 M 개

필터 인식 Vamana:

각 점 p 의 이웃 = 거리/필터_상관성 혼합 기반

- 같은 필터 그룹의 점들을 함께 연결

- 그룹 간 "다리" 역할 노드 유지

41.8 디스크 I/O 최적화

메모리 버퍼:

메모리 계층:

- 1 단계: GPU 캐시 (가장 자주 접근)
- 2 단계: RAM 버퍼 (핫 노드)
- 3 단계: SSD 캐시 (온도 낮은 노드)
- 4 단계: 메인 SSD (대부분의 데이터)

배치 I/O:

여러 쿼리의 필요한 벡터들을 함께 로드:

- 쿼리 1 → 노드 {1, 5, 12, 3}
- 쿼리 2 → 노드 {2, 5, 8, 3}
- 쿼리 3 → 노드 {1, 7, 12, 8}

공통 노드 {1, 5, 12, 3, 2, 8, 7}를 한 번에 로드

41.9 메모리 사용 분석

메모리 구성:

벡터 데이터: $N \times \text{dim} \times 4$ 바이트 (SSD 에 저장)

메타그래프: $N \times M \times 4$ 바이트 (메모리, $M=64$)

= 1M 벡터, $M=64$: 256MB

속성 인덱스: 속성 수 $\times$ (인덱스 구조 크기)

= 100 개 속성, 단순 비트맵: 12.5MB/속성

41.10 필터 조건의 복잡성 처리

단순 필터:

```
price > 100  
color == "red"
```

복합 필터:

```
(price > 100 AND price < 500) AND  
(color == "red" OR color == "blue") AND  
(stock > 0)
```

평가 순서 최적화:

1. 가장 선택적인 조건부터 평가
2. 조기 종료 (AND 에서 첫 거짓)
3. 속성 인덱스 활용 (범위 쿼리)

41.11 비교: Filtered-DiskANN vs Exqutor

측면	Filtered-DiskANN	Exqutor
그래프	Vamana	HNSW
저장소	디스크 기반	메모리 기반
필터 처리	동적 범위 조정	필터-우선 또는 하이브리드
특화	대규모 배치	실시간 쿼리
메모리	매우 효율적	중간 정도
속도	중간~높음	매우 높음

41.12 Exqutor 에의 시사점

필터 선택도 적응 메커니즘:

- 동적으로 필터 선택도를 추정
- 선택도에 따라 검색 범위 조정
- 필터 조건이 매우 선택적일 때: 광범위 검색 필요

속성 인덱싱 전략:

- 자주 함께 필터링되는 속성들을 함께 저장
- 범위 필터 지원을 위한 B-트리 구조
- 메모리와 성능 사이의 균형

진입점 최적화:

- 필터를 만족하는 진입점을 빠르게 찾기
 - 필터별 진입점 캐싱의 효과
-

추가 제기 문제

1. **필터 선택도의 사전 예측:** 주어진 필터 조건에 대한 선택도를 쿼리 전에 정확하게 예측할 수 있을까?
통계 기반 모델의 효율성은?
2. **다중 필터의 동시 최적화:** 여러 속성에 대한 필터 조건이 있을 때, 평가 순서를 동적으로 최적화하면 성능을 얼마나 향상시킬 수 있을까?
3. **HNSW와 필터링의 결합:** Filtered-DiskANN의 접근 방식을 HNSW(Exqutor가 사용)에 적용했을 때의 성능 특성은?
4. **필터 통계의 유지:** 인덱스가 동적으로 변할 때 필터 선택도 통계를 어떻게 효율적으로 유지할 것인가?
5. **비용 모델:** 필터 평가 비용과 거리 계산 비용을 모두 고려한 종합적 쿼리 최적화 모델은?
6. **캐시 친화성:** 필터 조건을 만족하는 벡터들이 메모리상에서 인접하도록 배치하면 성능을 얼마나 향상시킬 수 있을까?
7. **필터 인덱스 압축:** 속성 비트맵을 압축하면서도 빠른 필터 평가를 유지할 수 있을까?